UMJI FA22 VG101 Lab 5: Number Guessing Game - All in One Version

Author: Zijun Yang. Please email zijunyang@sjtu.edu.cn if you find any error or ambiguity in this file.

Besides the `.pdf` file, a `numberGuessingGameAllInOne.html` is provided which preserves the original format of the description to the greatest extent.

Brief Introduction to the Game

The AB Number Guessing Game is played with two people. One person chooses a 4-digit number in which no digit is repeated, say X, and the other person tries to guess it in as few turns as possible. After the guesser guesses some number, say Y, the other person gives information on how closely Y matched with X:

- If some digit in Y coincides with a digit in X (in the same position), then it counts an `A`.
- If some digit in Y exists in X but is in the wrong place, then it counts a `B`.

The guesser should be informed of how many `A` and `B` he/she can get.

For instance, if X is 0123 (yes, starting with 0 is valid in this game) and we guess 1263, we get `1A` and `2B`, because the 3 is in the right place and the 1 and 2 are the right numbers but not in the right place.

This proceeds until the guesser gets the exact number.

If you are still confusing about the procedure of the game, check out this page by coming up with a number in your mind and letting the bot guess your number.

Background and Goals of the Lab

Bob is playing this game with Alice. He finds it quite hard to get the exact number. He reaches out for your help hoping that you can make a program that can record the game and give him some suggestions on which number he should guess.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 5943
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
```

At the beginning of every turn, Bob wants to know:

- How many turns of guessing this is. This is shown as `Turn #1.` above.
- How many numbers are still possible to be the correct answer. This is shown as `The number of possible answer is 5040.` above.
- Which number the smart AI suggests him to guess. This is shown as `AI's choice is to guess 5943` above.

The program should then ask Bob to guess a number, **and if Bob decides to use the AI's choice**, he can simply press `enter`, without inputting the number.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 5943
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  1234
Input the result of the new guess 1234:
   0 2
```

If Bob wants to revert to the last turn he can input `-`.

After Bob guesses a number, Alice should give him the information. The program should require Bob to input the information. For example, if Alice's number is 2468, then Bob gets `0A` and `2B`, which means he should input `0 2` to the program.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 5943
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  1234
Input the result of the new guess 1234:
   0 2
------------------------------
Turn #2. The number of possible answer is 1260. AI's choice is to guess 2450
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
```

After Bob inputs the comparing result, a new turn starts...

End of the Program

The program ends in two situations:

- If Bob guesses the exact number, i.e. the comparing result is `4A 0B`, the program should end with an information showing how many times he has guessed.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 0697
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 0697:
   2 0
------------------------------
Turn #2. The number of possible answer is 180. AI's choice is to guess 2597
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 2597:
   1 0
------------------------------
Turn #3. The number of possible answer is 48. AI's choice is to guess 8617
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 8617:
   2 1
------------------------------
Turn #4. The number of possible answer is 6. AI's choice is to guess 8691
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 8691:
   0 2
------------------------------
Turn #5. The number of possible answer is 1. AI's choice is to guess 0817
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 0817:
   4 0
------------------------------
Congratulations! You've guessed the correct number in 5 steps!
```

- If there's something wrong with the information given by Alice, the number of possible answers can be 0. In this case, the program should end with an information showing that something is wrong.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 1687
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 1687:
   0 0
------------------------------
Turn #2. The number of possible answer is 360. AI's choice is to guess 9023
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 9023:
   0 0
------------------------------
Oops! It seems that there's something wrong with the input, since a number
satisfying such input does not exist.
```

Safe Input

Bob is careless. He once inputted `1233` as the number he guessed into the program and led the program to crash. He wants you to write a safe input mechanism to ask him to reinput if he inputs something wrong.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 8761
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  -1234
Your guess is invalid. Input again:
   7777
Your guess is invalid. Input again:
   3212
Your guess is invalid. Input again:
   1234
Input the result of the new guess 1234:
   0 5
Your result is invalid. Input again:
  -1 2
Your result is invalid. Input again:
```

Bob is not that careless, however, and he promises that all the inputs will be integer.

Overview

To finish this lab, you are expected to write about 130 lines of code. This is not a big amount, but also not a small one, so we give you plenty of time - it is always wise to start early.

`utils.c`

Implement the following functions in `utils.c`:

- `getDigit`
- `isValidNum`
- `isValidResult`
- `printNum`
- `printWithWidth`

`Game.c`

Implement the following functions in `Game.c`:

- `initGame`
- `judgeGuess`
- `updateGameWithLastTurn`
- `takeGuessResult`
- `revert1Turn`
- `makeGuess`

`terminal.c`

Implement the following functions in `terminal.c`:

- `requestNewGuess`
- `requestNewGuessResult`
- `endGame`

Rubrics

75% of the total score of this lab is about correctly implementing the functions. For this part, you should submit your code on JOJ. The testcases test every function you implemente and the score of each function is equal.

There are 14 functions needed to be implemented and thus there will be 14 test cases on JOJ. For instance, you get 10 of the testcases correct, you will earn $75\% \times \frac{10}{14} = 53.57\%$ of the total score of this lab. Only if you attend the lab, you can get this part of the score.

25% of the total score of this lab is about attendance. You should have a conversation with the TA to discuss your code.

Full Feature Demonstration

The following two interaction records contain every function of the lab. If you are not sure about the output format of a certain function, you can always use this demonstration as a reference.

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 7146
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  -
Revert to last turn.
This is turn #1. Cannot revert.
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 0945
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  1111
Your guess is invalid. Input again:
   1234
Input the result of the new guess 1234:
   2 3
Your result is invalid. Input again:
  2 2
------------------------------
Turn #2. The number of possible answer is 6. AI's choice is to guess 1243
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  -
Revert to last turn.
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 9217
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
Input the result of the new guess 9217:
   4 0
------------------------------
Congratulations! You've guessed the correct number in 1 steps!
```

```
-----------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 7298
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  1234
Input the result of the new guess 1234:
   0 0
-----------------------------
Turn #2. The number of possible answer is 360. AI's choice is to guess 6507
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
  5678
Input the result of the new guess 5678:
   0 0
-----------------------------
Oops! It seems that there's something wrong with the input, since a number
satisfying such input does not exist.
```

Tasks in Details

Implement `getDigit`

This function accepts two parameters, a 4-digit integer `num` and an integer `digitPos` representing the position of the digit starting from the highest digit (from the left). Note that if `num` is less than 4-digit, 0 will be filled at the beginning of the number to make it a 4-digit one.

The function returns the digit at the position of `digitPos` in the integer `num`.

For instance,

```
getDigit(23,1)  returns 0
getDigit(23,2)  returns 0
getDigit(23,3)  returns 2
getDigit(23,4)  returns 3
getDigit(4321,2) returns 3
getDigit(123,1)  returns 0
getDigit(123,4)  returns 3
```

Implement `isValidNum`

The function accepts one integer `num` which is the number that is going to be judged.

To be a valid number, `num` must be a 4-digit number in which no digit is repeated.

Note that if the number is less than 4-digit, 0 will be filled at the beginning of the number to make it a 4-digit one.

The function returns `true` if `num` is valid and `false` if not.

For instance,

```
isValidNum(-1234) returns false
isValidNum(1123) returns false
isValidNum(1234) returns true
isValidNum(123)  returns true
isValidNum(12)  returns false
isValidNum(10000) returns false
```

Implement `isValidResult`

The function accepts one `Result` struct `res` as parameter and returns whether `res` can be a valid comparing result.

You can find how `Result` struct is defined by hovering your cursor on the word `Result` or by checking out its definition in `Game.h`.

For instance,

```
isValidResult((Result){3,1}) returns false
isValidResult((Result){2,3}) returns false
isValidResult((Result){-1,1}) returns false
isValidResult((Result){0,0}) returns true
```

Implement `printNum`

The function accepts one 4-digit integer `num` as parameter and prints the number into 4-digit format onto the screen. `num` is guaranteed to be a valid number.

Note that if the number is less than 4-digit, 0 will be filled at the beginning of the number to make it a 4-digit one.

For instance,

```
printNum(1234) prints 1234
printNum(123) prints 0123
```

*Hint: try to use functions you have implemented.

Implement `printWithWidth`

The function accepts a char array `s` and an integer `width` as parameters.

It will first print `s` onto the screen, and if the length of `s` is less than `width`, it will print space until the width of the string printed by the function is equal to `width`.

For instance, (space will be replaced by `_` in the following example for better readability, but just use space when you are implementing the function)

```
printWithWidth("0123456789012345678",20) prints:
0123456789012345678_
printWithWidth("01234567890123456",20)  prints:
01234567890123456___
printWithWidth("012345678901234567890",20) prints:
012345678901234567890
```

Tips for the Following Implementation

If you are confused about the specific format, check out the section "Full Feature Demonstration" above.

Understand `Game` struct

You can find the declaration of the `Game` struct in `Game.h` header file. All the variables in this abstract data type are commented. We think `num[]` is probably the most confusing one to understand so we are going to explain more about it here.

It is obvious that not all the 4-digit number is valid. Number like 1223 has a repeated digit 2. The integer array `num[]` stores all the valid number in an ascending order from index=0, i.e.

```
num[0]=123
num[1]=124
num[2]=125
...
```

You can think of this array as a map that links a valid number with an index.

`isPossible[]`, a boolean array, as commented in the header file, stores whether a number is possible to be the final answer given a set of guesses and comparing results.

Implement `initGame`

This function accepts a pointer to a `Game` struct `game` and initializes it.

Hint: if your initialization is correct, `possibleCnt` should be 5040, exactly equal MAX_POSSIBLE defined at the beginning of the header file, and `num[5039]` should be 9876, the max possible number.

Implement `judgeGuess`

This function accepts two integers, one is the correct number `correctNum` and the other is the guessed number `guessNum`.

The function should return the comparing result between the two numbers. You can find the comparing rules in the section "Brief Introduction to the Game" above.

For instance,

```
judgeGuess(1234, 1240) should return (Result){2, 1}
```

Implement `updateGameWithTurn`

This function accepts a pointer to a `Game` struct `game` and an integer `turn`. It updates `possibleCnt` and `isPossible[]` in `game` by the set of guess and result of `turn`.

That is to say, if `num[i]` does not satisfy `game->guesses[turn]` and `game->results[turn]`, `isPossible[i]` should be assigned `false`, meaning `num[i]` is impossible to be the final answer.

You do not need to update `game` with other set of guess and result in `game`.

Implement `takeGuessResult`

This function accepts a pointer to a `Game` struct `game`, an integer `guess` representing the guessed number, and a `Result` struct `result` representing the comparing result.

The function updates `game` given `guess` and `result`, and then returns the status of the game.

Implement `revert1Turn`

This function accepts a pointer to a `Game` struct `game` and reverts `game` by 1 turn, i.e. delete the last set of `guess` and `result`.

If there's no `guess` and `result`, i.e. `turnCnt == 0`, print `This is turn #1. Cannot revert.` onto the screen with a new line at the end.

Implement `makeGuess`

This function accepts a pointer to a `Game` struct `game`, and returns an integer, representing the number suggested by AI.

To lighten your workload, **we will use a unified strategy** that is to pick a random number that is still possible to be the final answer.

Check out references for srand() and rand() to generate random numbers.

> If you are interested in the best AI algorithm of this game, check out this video by 3Blue1Brown which might give you some inspiration. The webpage shown in the lab description is an implementation of that algorithm and can guess the answer in an average of 5.3 turns.

Implement `beforeTurn`

We have implemented this function for you, so you don't need to write anything. You're encouraged to read the codes and to figure out how this function works.

This function accepts a pointer to a `Game` struct `game` and is responsible for printing the following information onto the screen:

```
------------------------------
Turn #1. The number of possible answer is 5040. AI's choice is to guess 3716

```

Notice that there's a new line at the end.

The turn count, possible number count and AI's choice is determined by `game`.

The function returns an integer representing the number AI chooses.

Implement `requestNewGuess`

The function accepts an integer `aiGuess` which is the AI's choice in this turn.

It is responsible for printing the following information onto the screen:

```
Input a guess, press enter to take AI's choice, or input '-' to revert 1 turn:
```

Notice that there are some spaces after the string, and together they have a width of 85.

This function handles what the user choose to guess and returns the number the user choose.

If the user inputs some invalid numbers, the function should keep asking the user to reinput the number until it is valid.

```
Your guess is invalid. Input again:
```

Notice that there are some spaces after the string, and together they have a width of 85.

If the user inputs nothing but directly presses the enter, the function should take the AI's choice as user's choice.

If the user inputs `-`, print `Revert to last turn.` with a new line at the end and return `-1` (which means that the user wants to revert 1 turn).

Implement `requestNewGuessResult`

The function accepts one parameter `guess` which corresponds to the comparing result requested by this function.

The function handles the input of the comparing result. It should print the following information onto the screen:

```
Input the result of the new guess 1687:
```

Notice that there are some spaces after the string, and together they have a width of 85.

Bob guarantees that he will input two integers at a time. If the comparing result inputted is valid, the function should return it. If it is not, the function should print the following information and keep asking the user to reinput the comparing result until it is valid:

```
Your result is invalid. Input again:
```

Notice that there are some spaces after the string, and together they have a width of 85.

Implement `endGame`

This function accepts two parameters, a pointer to a `Game` struct `game` and an `enum GameResult gameResult`.

The function is called when the game is end and is responsible for displaying some information.

If the Bob guessed the correct number, it should display:

```
------------------------------
Congratulations! You've guessed the correct number in 1 steps!

```

Notice that there's a new line at the end.

The turn count is determined by `game`.

If there's something wrong with the inputs and thus there does not exist a number satisfying all the inputs, it should display:

```
------------------------------
Oops! It seems that there's something wrong with the input, since a number
satisfying such input does not exist.

```

Notice that there's a new line at the end.

Reference

Demo Project 2 Description VG101 SU 2022